Model Selection & Regularization

Oct. 14, 2025 - Lecture 5

Outline

- Bias-variance tradeoff
- Model selection
  - Performance measures
- Cross-validation
- Feature construction
- Feature selection (Subset Selection, Shrinkage / Regularization, Dimension Reduction)
- Ridge, LASSO, ElasticNet

Predictions

So far, we've talked about **training models** to make **predictions**. We've minimized the training error and hoped that the model will perform well on new data.

Since we train models to make **predictions on unseen data** how do we know if our model will **generalize**? Is training error a good enough proxy for how the model will actually perform?

(Generalization: the ability of a trained model to accurately make predictions on new, unseen data.)

Test Error

The expected test error can be broken down into:

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Bias = difference between the average prediction and the true value.

$$\text{Bias} = E[\hat{f}(x_i)] - f(x_i)$$

Variance = how much, on average, predictions vary for a given data point.

$$\text{Variance} = E[\hat{f}(x_i) - E[f(x_i)]]^2$$

Irreducible Error (ϵ) = noise we can't remove.

Bias

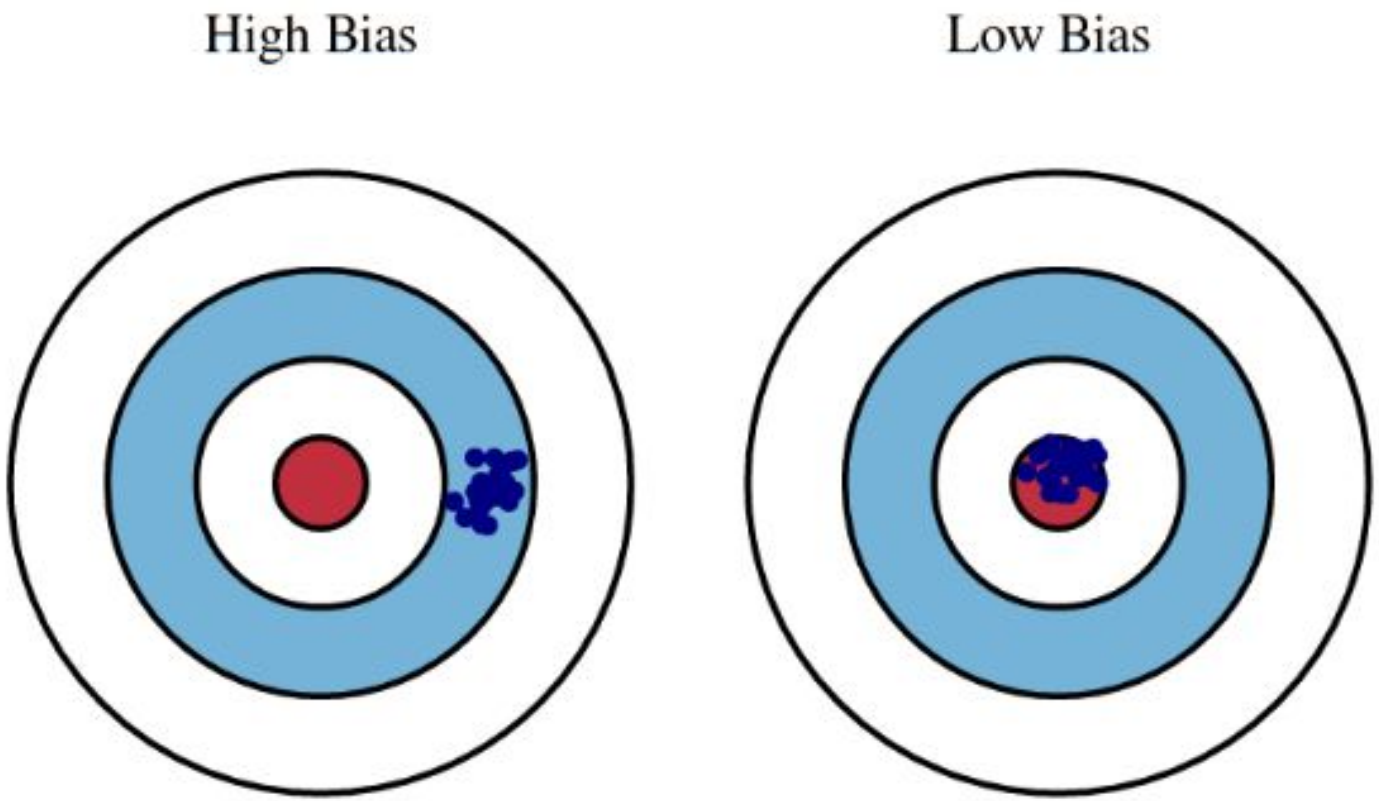
Bias = difference between the average prediction and the true value

$$\text{Bias} = E[\hat{f}(x_i)] - f(x_i)$$

Bias is the difference between the **average prediction** of our model ($E[\hat{f}(x_i)]$) and the true value we are trying to predict ($f(x_i)$).

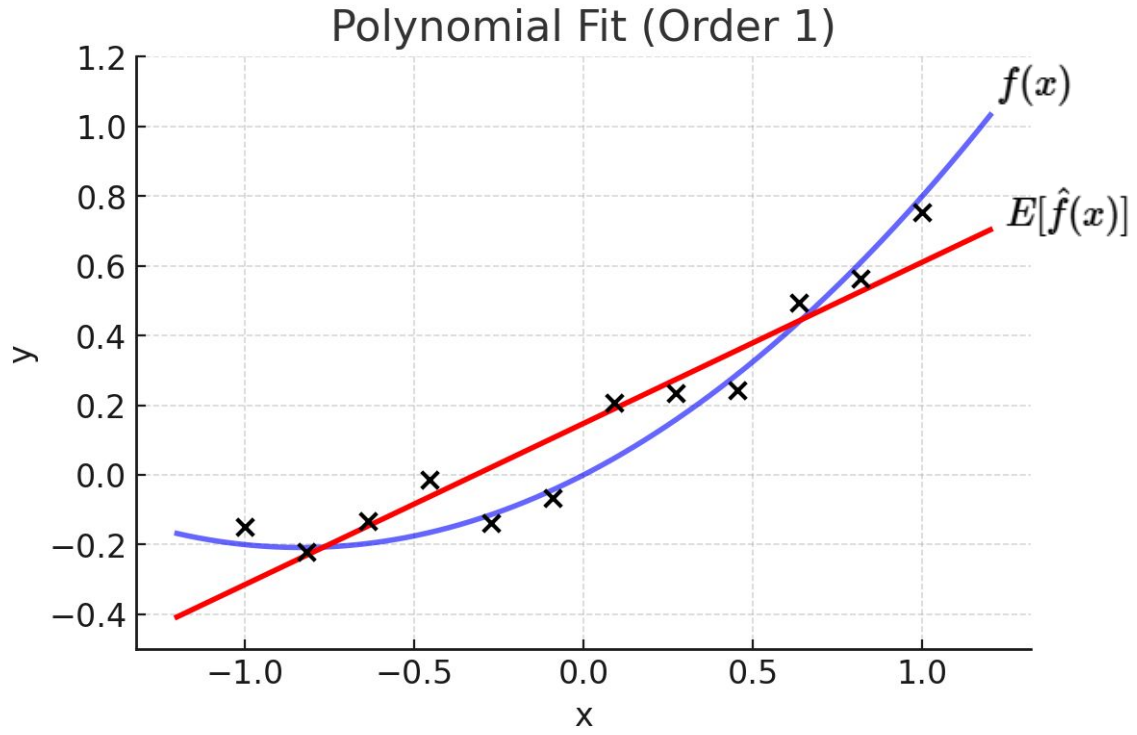
Bias is a **systematic error** that occurs when the true model and the fitted model are not of the same class. As model **complexity increases, bias decreases.**

Bias: Visual Example



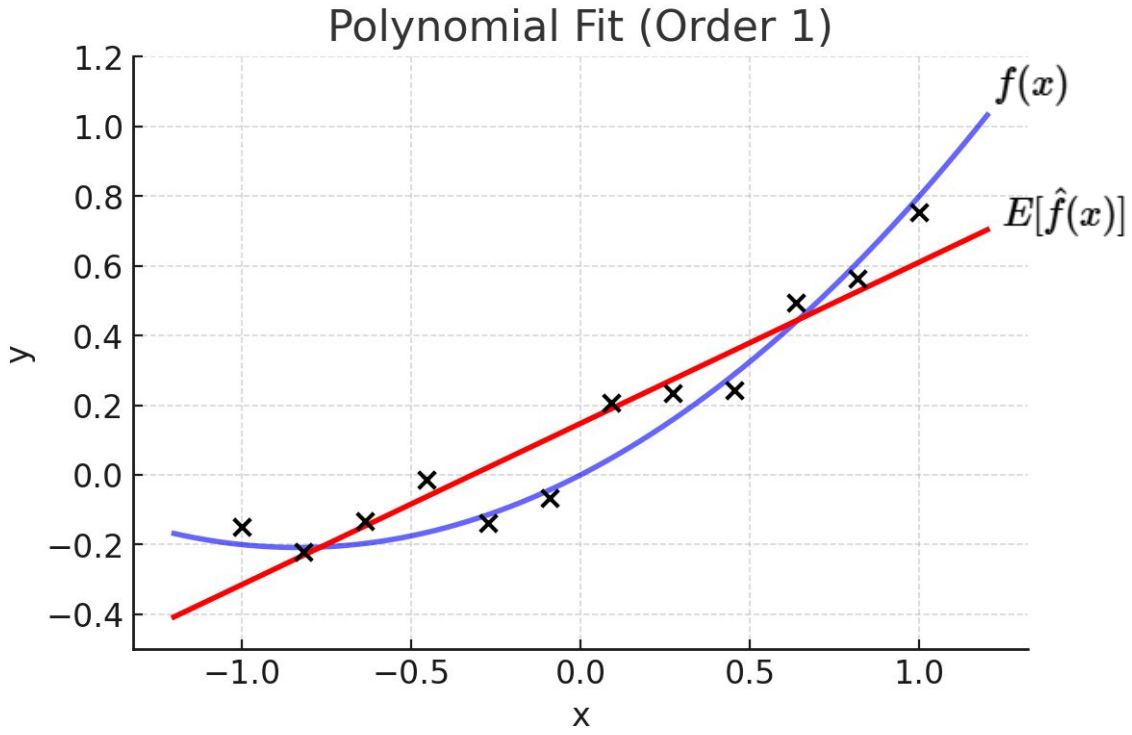
Bias: Example

What happens when we use a linear regression model if the data has a nonlinear relationship?



Underfitting (High Bias)

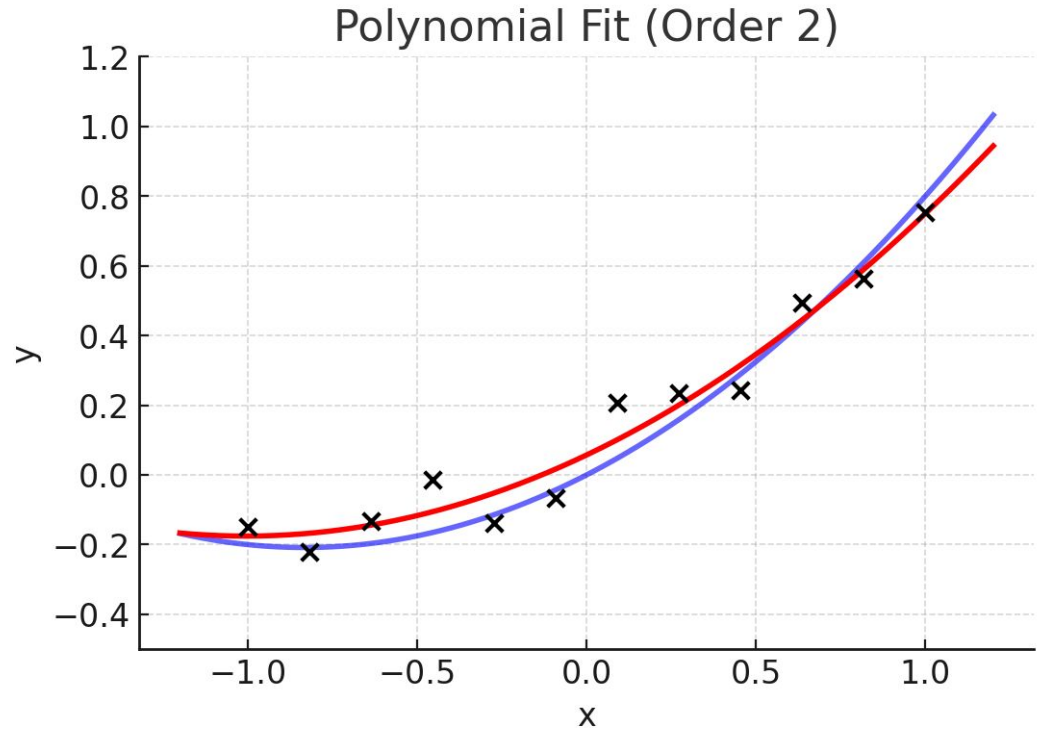
What happens when we use a linear regression model if the data has a nonlinear relationship?



How Can We Reduce Bias?

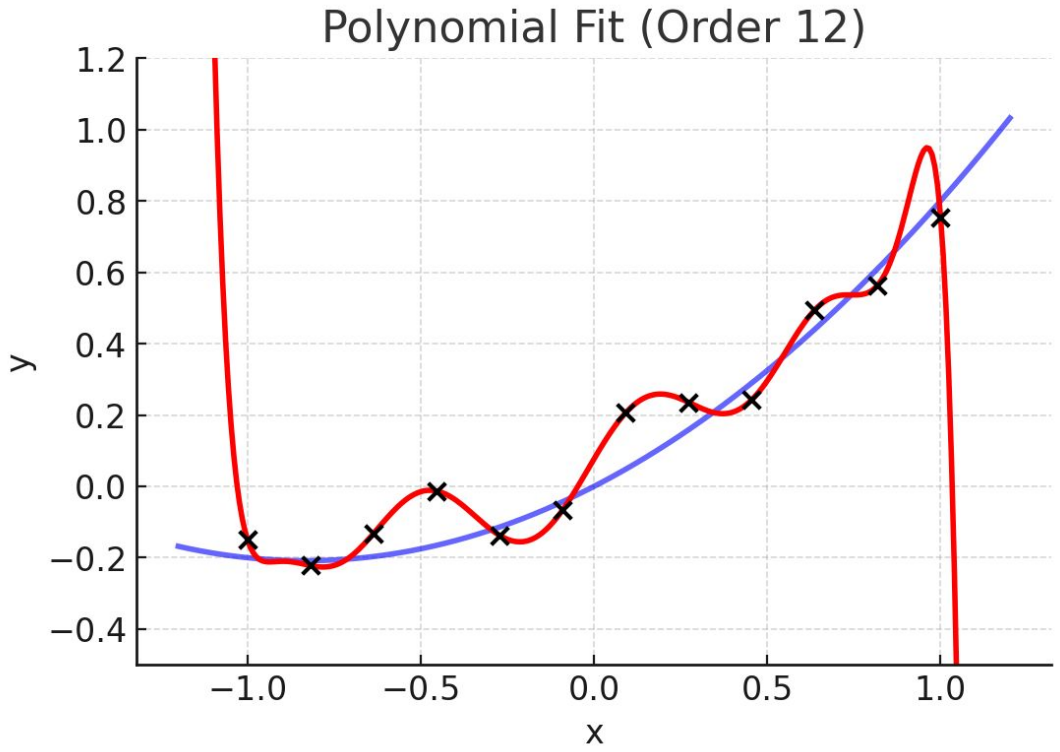
We can reduce bias by using a **more complex model.**

Example: An order 2 fit looks much better for this data.



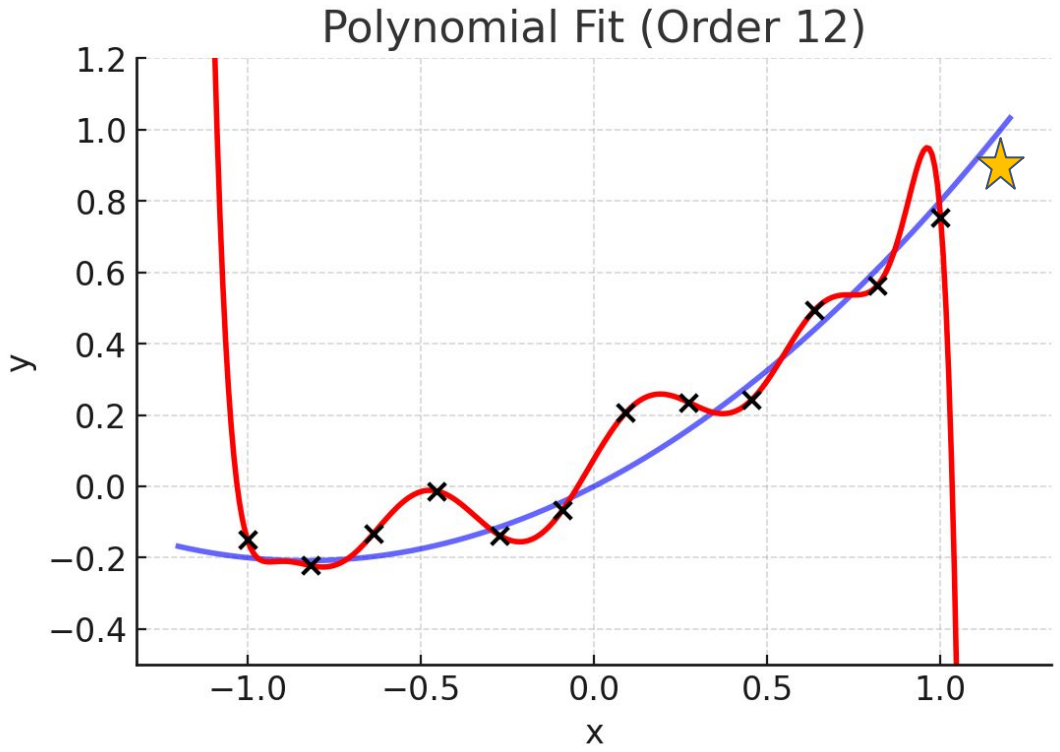
More Complexity

What if we keep increasing the model complexity? Isn't this a "perfect fit"?



More Complexity

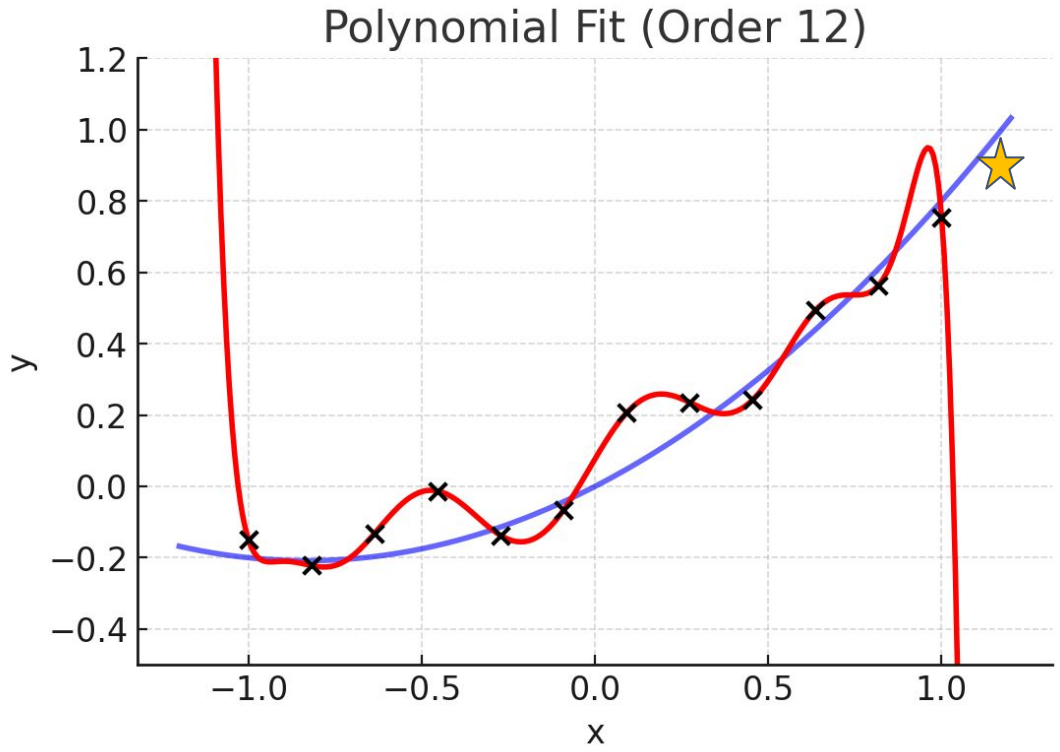
What if we have new data point?



The model will fail to predict this!

Overfitting (High Variance)

What if we have new data point?



The model will fail to predict this!

Variance

Variance = how much, on average, predictions vary for a given data point.

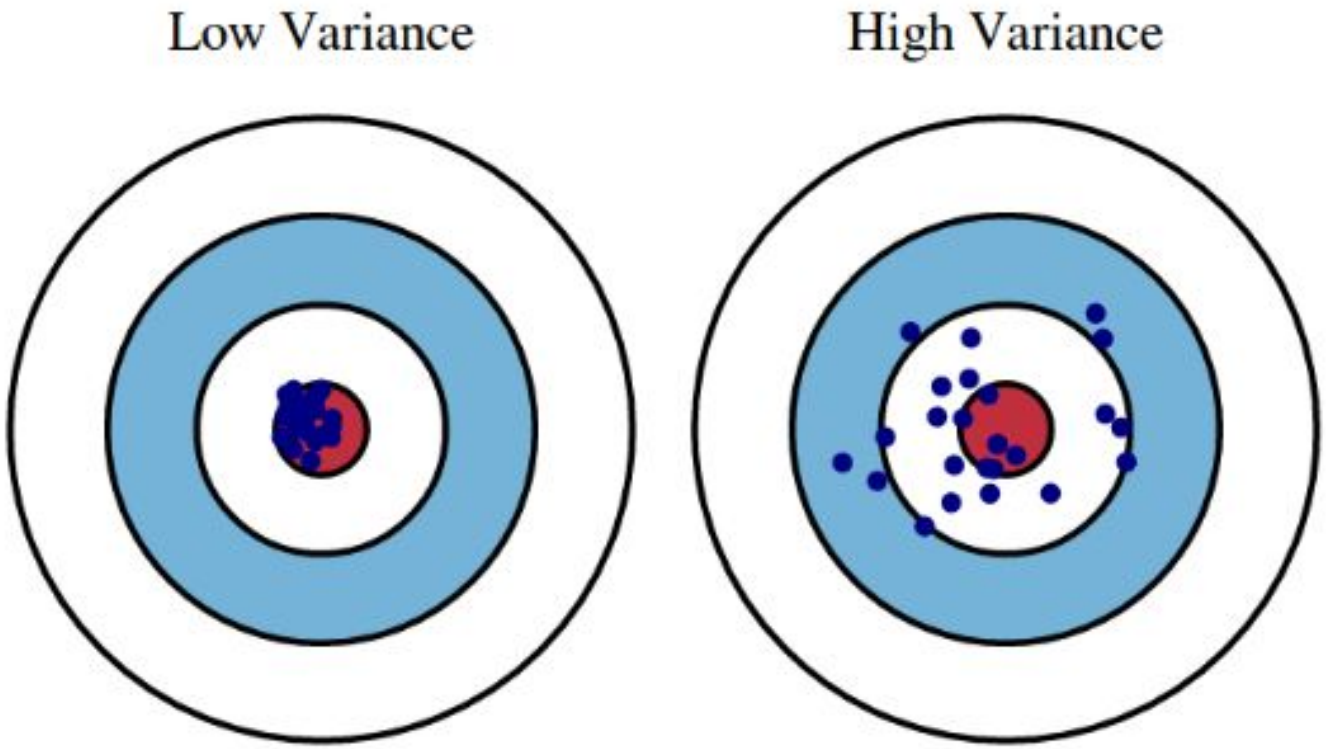
$$\text{Variance} = E[\hat{f}(x_i) - E[f(x_i)]]^2$$

Variance is the **measure of spread** in data from its mean position.

- The amount by which the performance of a model changes when it is trained on different subsets of the training data.

Models with **high variance** pay a lot of attention to training data and **do not generalize well**.

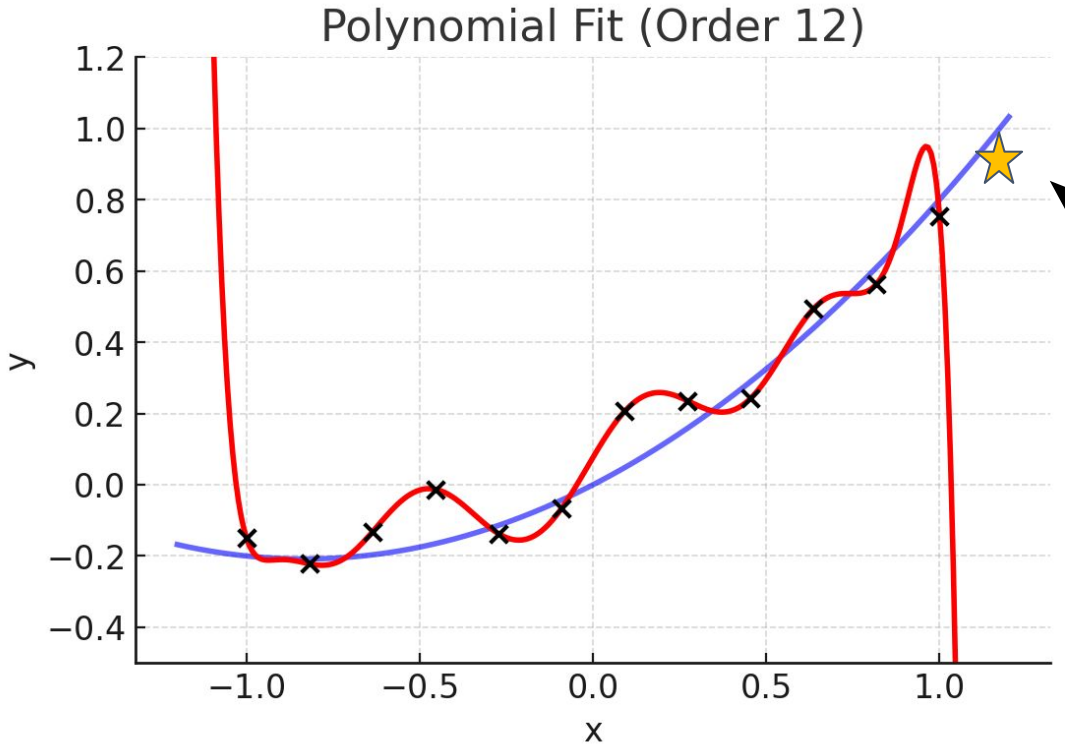
Variance



Variance (Example)

What happens when we use a complex model to fit data that has a simple pattern?

Overfitting. This model has **high variance.**



The model will fail to predict this!

Bias-Variance Tradeoff

When a model is **too simple** it ignores useful information, and the test error is mostly **from bias.**

When a model is **too complex**, it memorizes non-general patterns, and the error is mostly **from variance.**

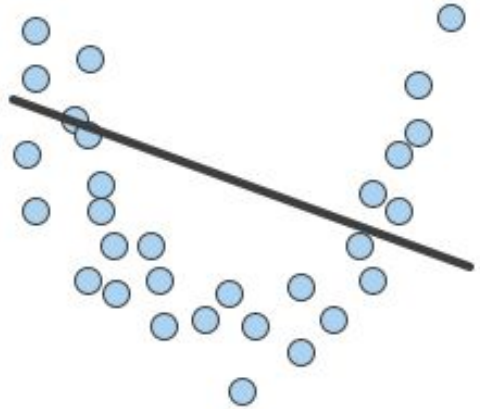
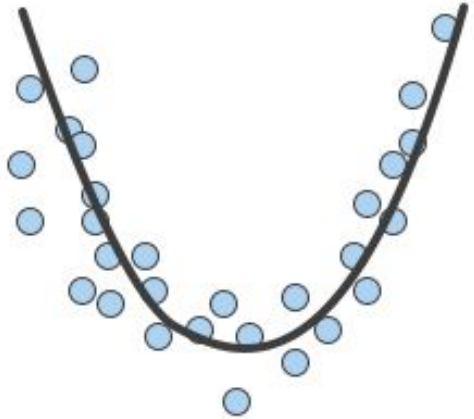
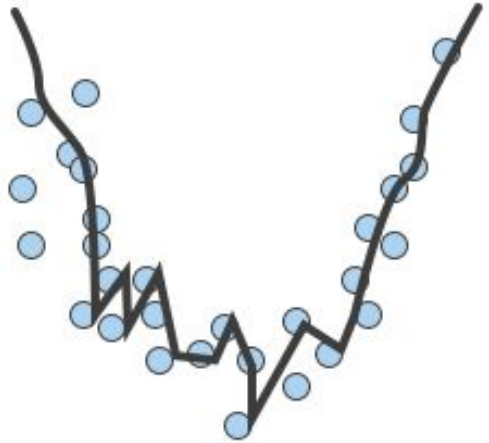
The **ideal model minimizes both bias and variance** to achieve the minimum error.

Bias-Variance Tradeoff (Technical Description)

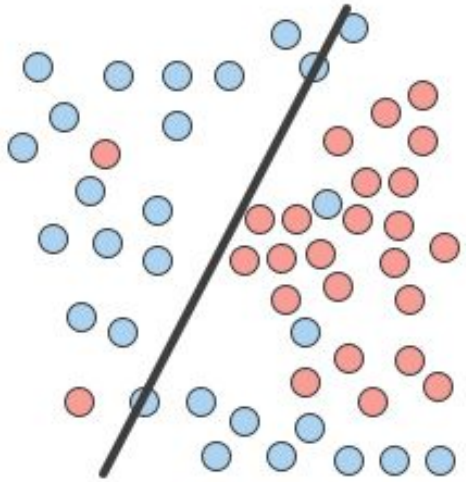
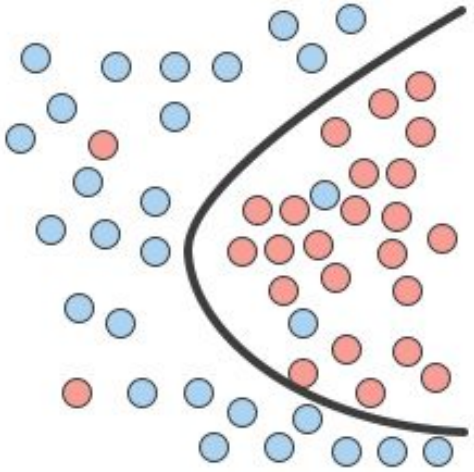
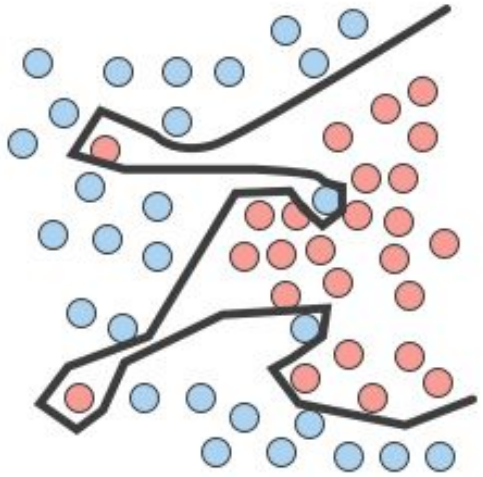
In general, as the number of tunable parameters in a model increase, it becomes more flexible, and can better fit a training data set. That is, the model has lower error or lower bias.

However, for **more flexible** models, there will tend to be **greater variance** to the model fit each time we take a set of samples to create a new training data set. It is said that there is greater variance in the model's estimated parameters.

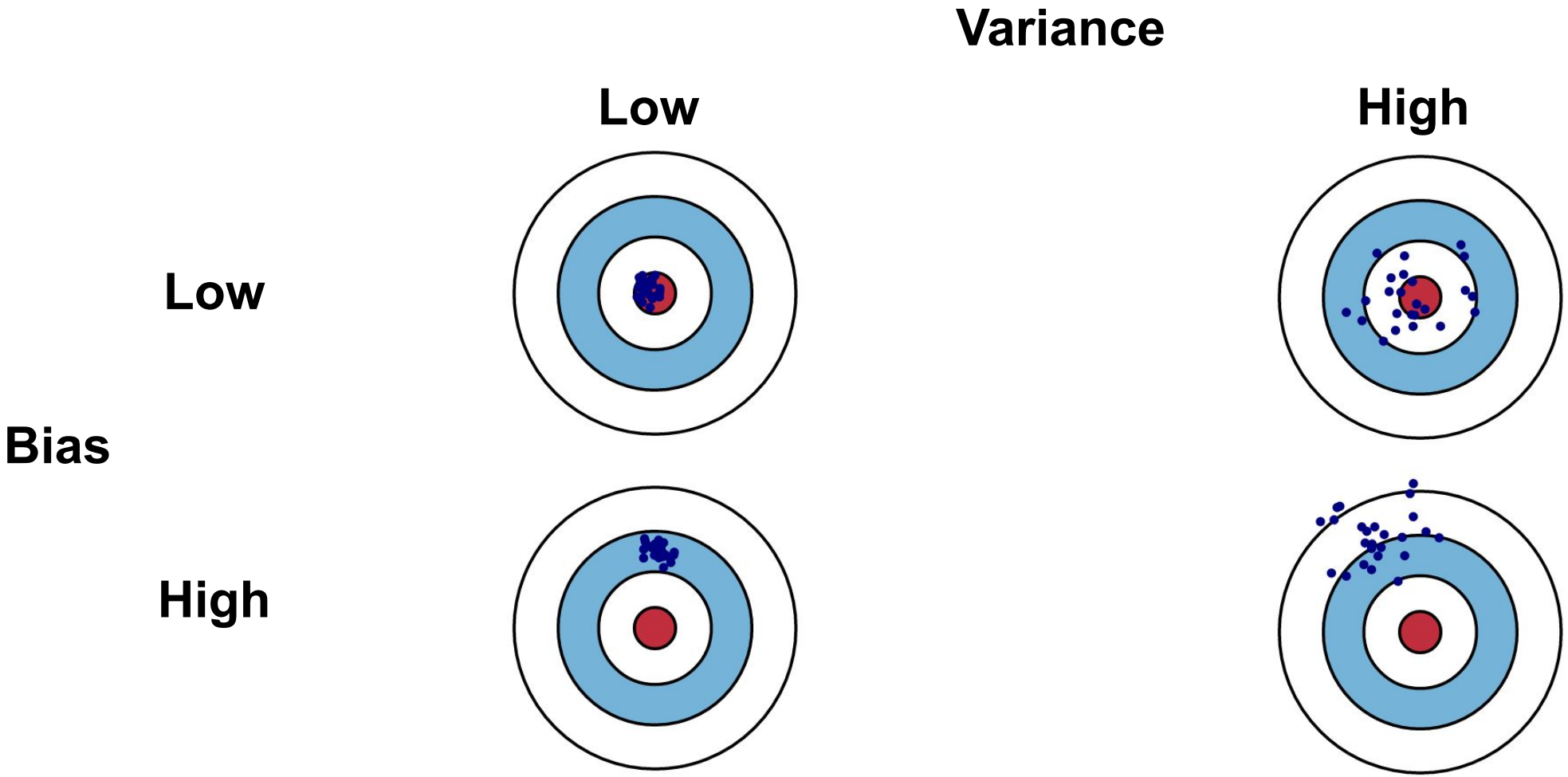
Underfitting & Overfitting

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration			

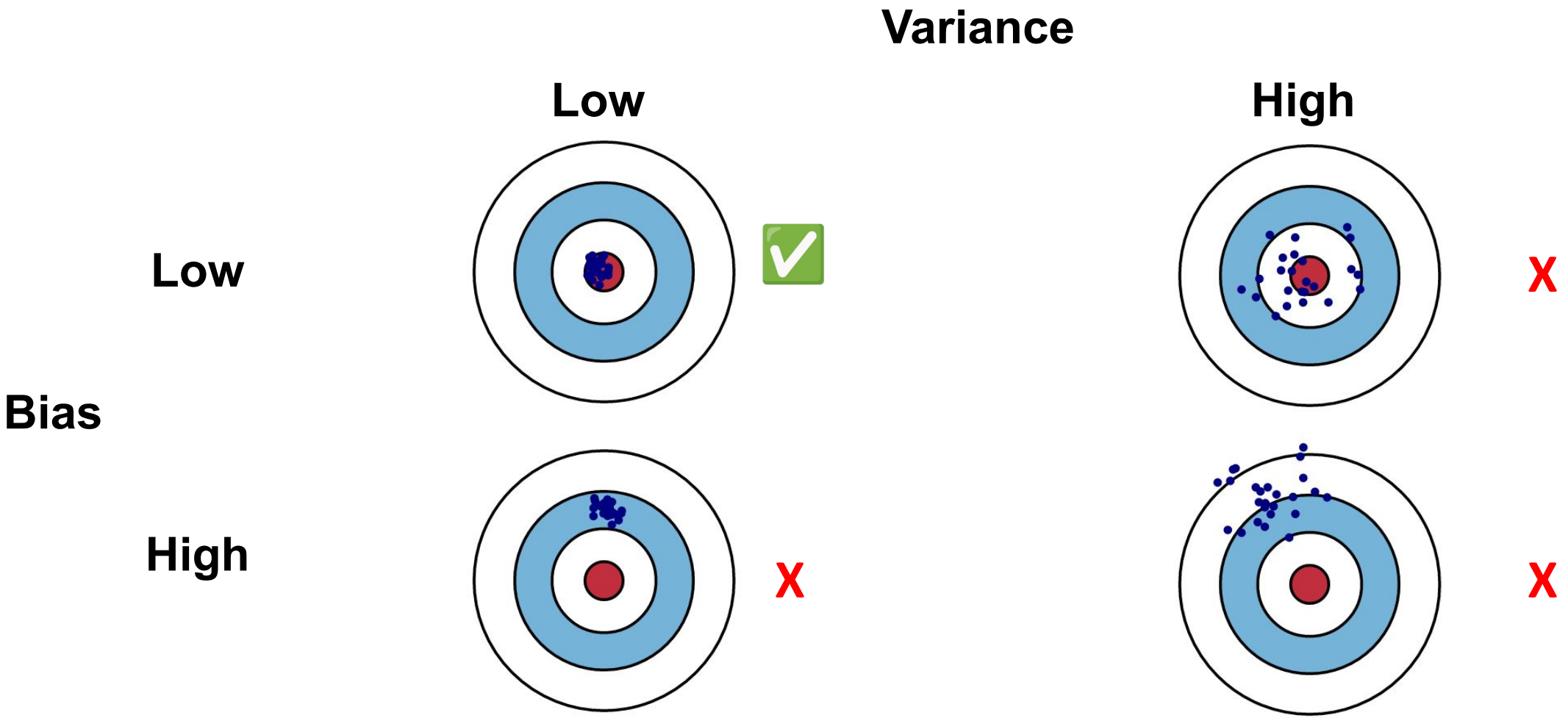
Underfitting & Overfitting

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Classification illustration			

Bias-Variance Tradeoff



Bias-Variance Tradeoff

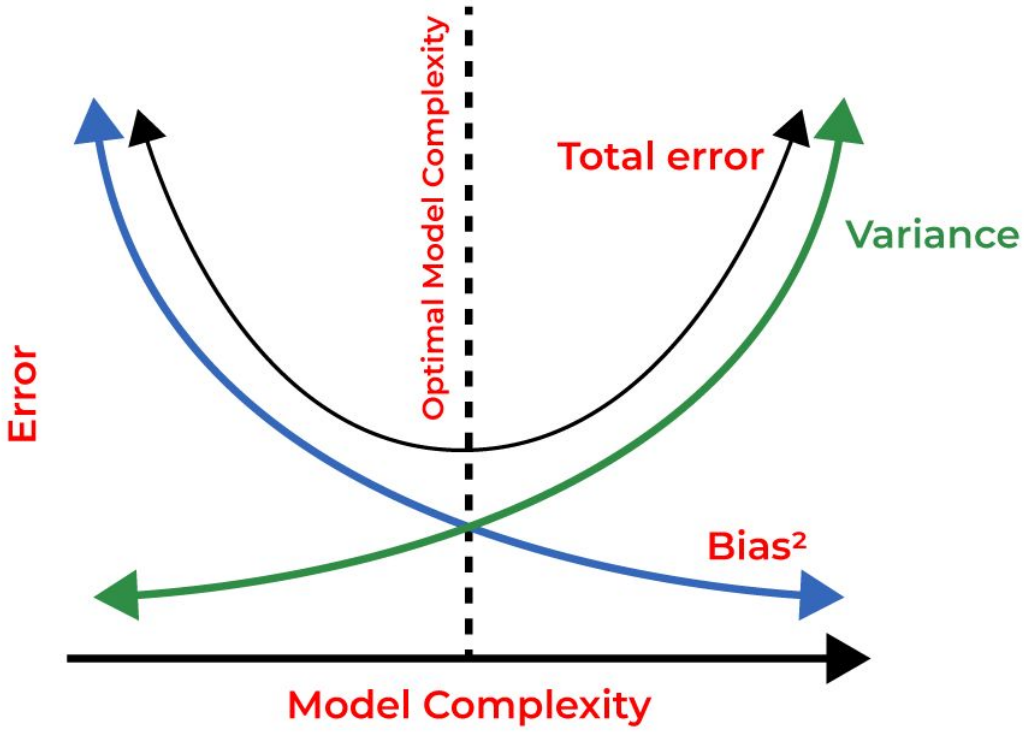


Bias-Variance Tradeoff

Unfortunately, these are **conflicting goals.**

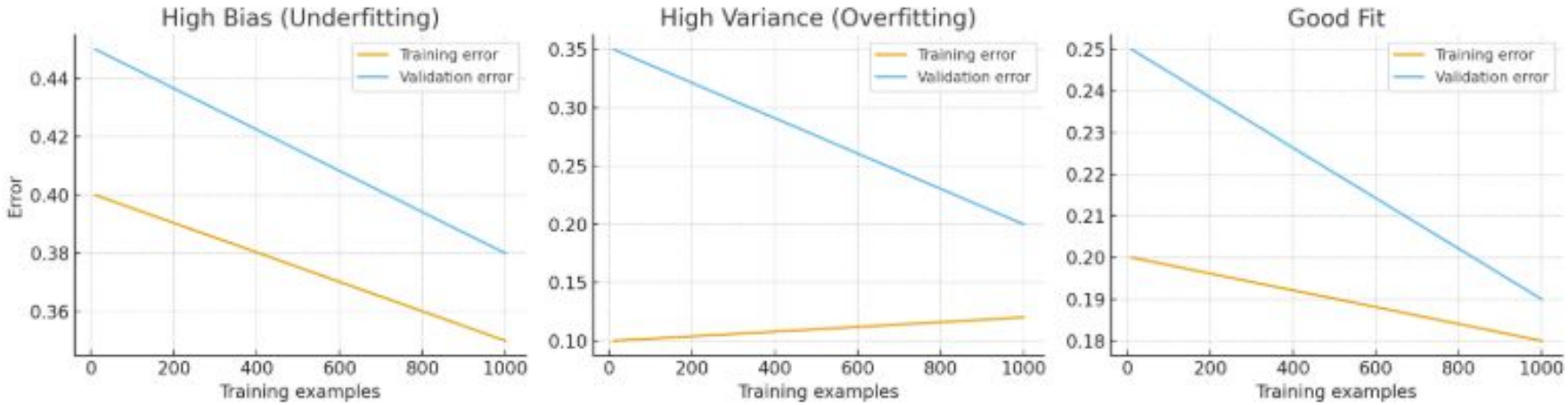
- As bias decreases, variance increases and vice versa.

So instead, we usually aim to find the model with the **optimal balance** between bias and variance.



Detecting Bias and Variance Using Learning Curves

Learning Curve: Plot training and validation error versus training set size.



Here you can see how the **training** and **validation** errors behave in each case:

- **High Bias (Underfitting):** Both errors are high and close together → the model is too simple.
- **High Variance (Overfitting):** Training error is low, but validation error is much higher → the model doesn't generalize well.
- **Good Fit:** Both errors are low and converge → the model generalizes appropriately.

Underfitting & Overfitting

Overfitting occurs when our model fits to the noise in the data**.** An overfit model will **fail to generalize.**

- While training error decreases, test error increases.

What Can We do About It?

Some ways that we can manage this tradeoff:

- Collect **more data** to reduce overfitting → may be unfeasible
- Use **cross-validation** to guide **model selection**
- Apply **regularization** (Ridge, LASSO, ElasticNet)
- Use **ensemble** methods (bagging, boosting) → next lecture

Cross-Validation

What is Cross-Validation (CV)?

Cross-validation (CV) is a technique used to check how well a machine learning model performs on unseen data while **preventing overfitting during model selection**. It works by:

- Splitting the dataset into several parts.
- Training the model on some parts and testing (validating) it on the remaining part.
- Repeating this resampling process multiple times by choosing different parts of the dataset.
- Averaging the results from each validation step to get the final performance.

Why do we Need Cross-Validation?

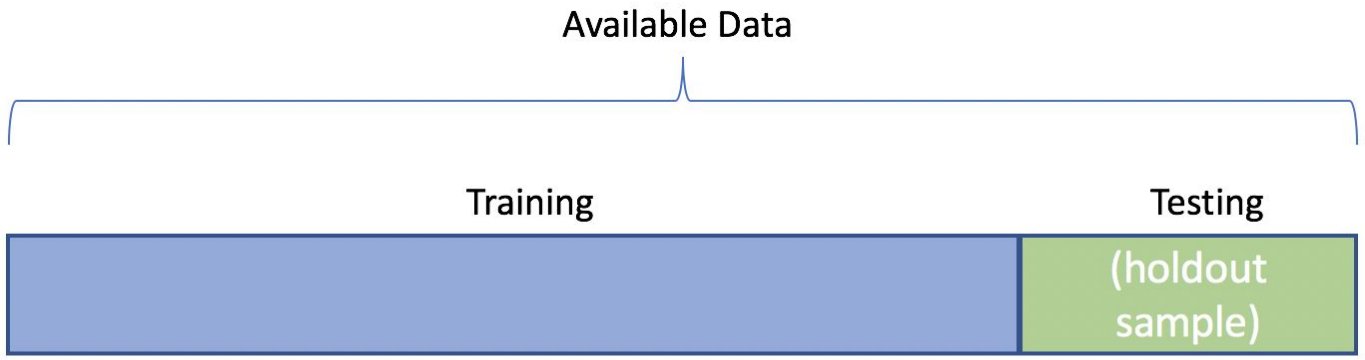
Cross-validation can be a safer option than just a train-test split.

A single train–test split can give **highly variable** results, depending on how the data is divided.

Train-Test Split

One way to test how our model will generalize is to create a **test set** by splitting off some of our data.

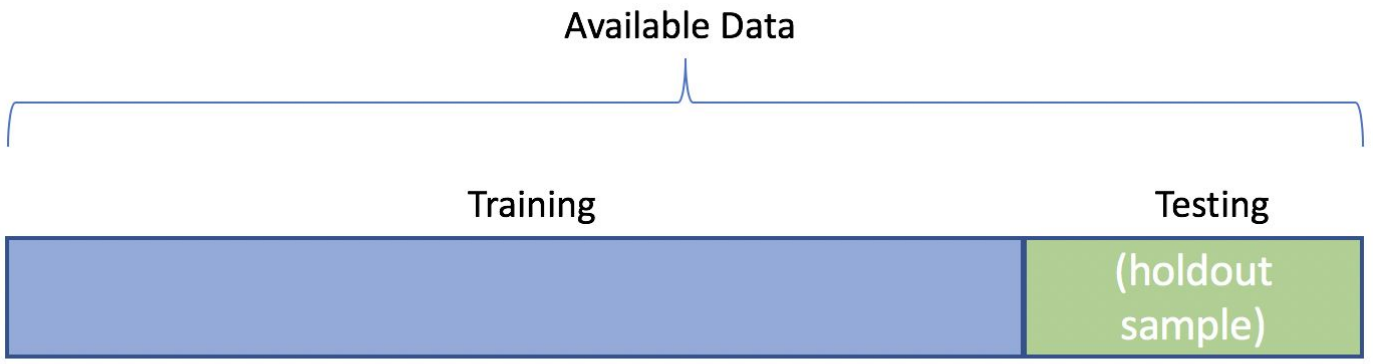
- The test set should be kept in a “vault,” and be brought out only at the end of the data analysis.



Train-Test Split: Test Set Size

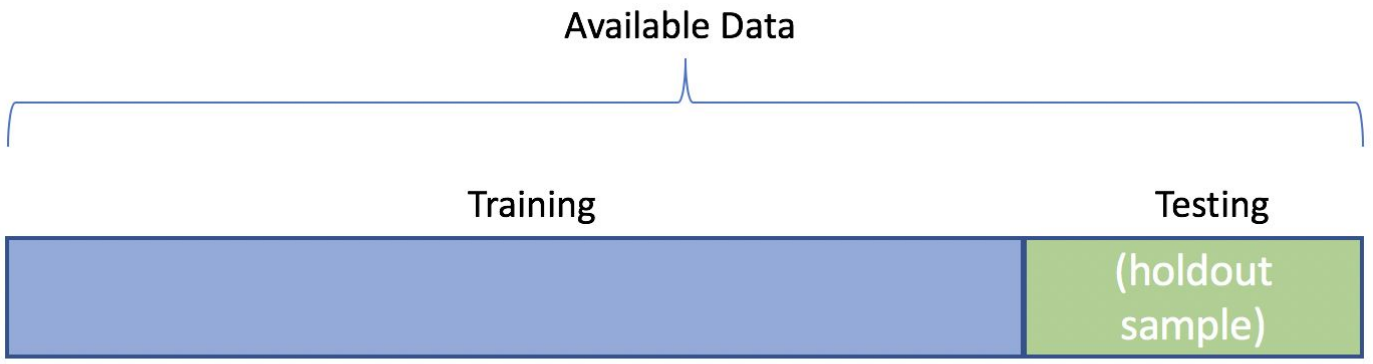
How big should our test set be? There's no perfect answer, typically between 5% and 20% of available data is used.

- Keep in mind, the more training examples we have, the better!



Train-Test Split: Disadvantages

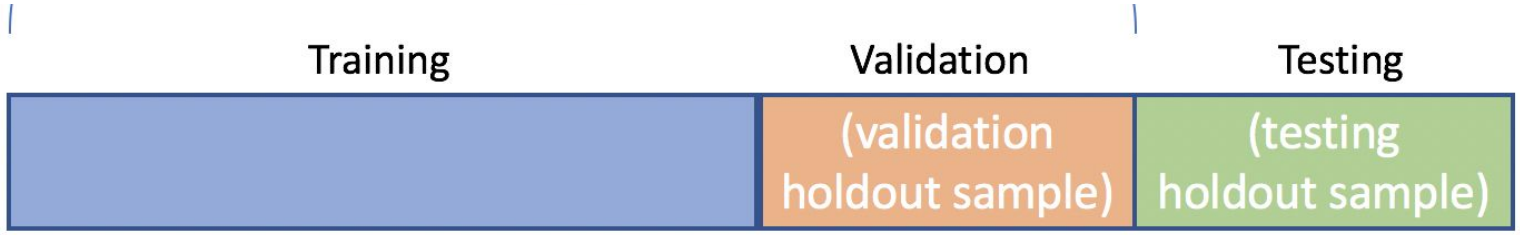
- By selecting the model that performs best on the test set, we risk **overfitting to the test data.**



Train-Test-Validation Split

To attempt to avoid overfitting to the test set, we can add a **validation set.**

1. The **training set** is used to **fit** the model parameters.
2. The **validation set** is used to **tune** hyperparameters or select between models.
3. A **test set** used to **estimate** the generalization error of the selected model.



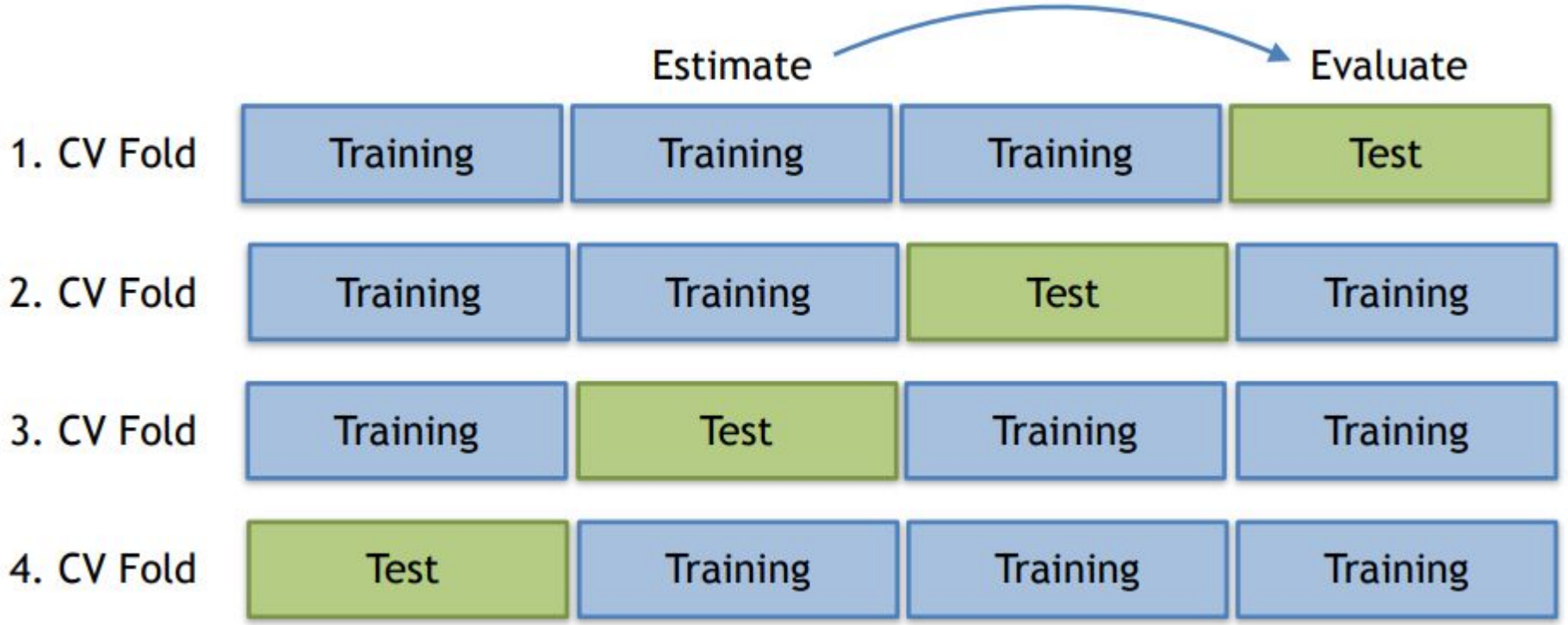
K-Fold Cross-Validation

We divide the data into K equal sized parts and train/validate on each:

1. Divide the data into k disjoint partitions of size n/k.
2. For each partition i = 1,...,k:
   a. Set aside partition i for evaluation.
   b. Train the model on all partitions except for partition i.
   c. Calculate the error on partition i.
3. Calculate the cross-validation (CV) error by averaging the error on the evaluation partitions.

K-Fold Cross-Validation Example: K = 4

We divide the data into 4 equal sized parts.



K-Fold Cross-Validation Example: K = n

When K is equal to the number of data points (n), this is a special case called **leave-one-out cross validation.**

- Only one data point is left out at a time.
- **Cons:** This can be time-consuming and unfeasible on very large datasets.

Model Selection

Suppose we train several models, how can we choose the "best" model?

Attempt 1) Choose the "best fitting" model on the training data.

This will always choose the most complex model. As we saw earlier, although complex models can achieve a "perfect fit" they may not be the "best" model since they are prone to overfitting.

Model Selection

Suppose we train several models, how can we choose the "best" model?

Attempt 2) Choose model with lowest test error.

Risk of overfitting to the test set (this is why we need CV)!

Model Selection

Suppose we train several models, how can we choose the "best" model?

Attempt 3) Choose model with lowest validation error.

This is the **strategy that is typically used in ML:** report the test error as a measure, but select the model with the **lowest validation error.**

What Can We do About It?

Some ways that we can manage this tradeoff:

- Collect **more data** to reduce overfitting → may be unfeasible
- Use **cross-validation** to guide **model selection**
- Apply **regularization** (Ridge, LASSO, ElasticNet)
- Use **ensemble** methods (bagging, boosting) → next lecture

What Can We do About It?

Some ways that we can manage this tradeoff:

- Collect **more data** to reduce overfitting → may be unfeasible
- Use **cross-validation** to guide **model selection**
- Apply **regularization** (Ridge, LASSO, ElasticNet)
- Use **ensemble** methods (bagging, boosting) → next lecture

Regularization

Feature Construction

How we can create, manipulate, rank and select relevant features from raw data?

Your data is extremely important to how well your model will perform. In the real world, it is often very, very difficult to get relevant, clean data ready for model training.

Feature Construction: Basis Functions

We can use **basis functions** to define new features:

$$\hat{y} = \sum_{m=1}^{M} \theta_m h_m(X)$$

Feature Construction: Basis Functions

Some common basis functions:

$h_m(X) = x_m, m = 1, 2, \ldots, p$ → Recovers the original linear model

$h_m(X) = x_m^2, h_m(X) = x_m x_{m+1}$ → Augment input with polynomial terms

$h_m(X) = log(x_m), h_m(X) = \sqrt{(x_m)}$ → Nonlinear transformations

Feature Selection

It is good practice to select the subset of features that are **most relevant to the problem.**

We also want our features to be **as independent as possible.**

Suppose we have 2 models with very similar performance, in general, we should favor the **simpler model.**

- **We can reduce generalization error and model complexity by removing irrelevant features or noise.**

Feature Selection Methods

There are several ways to choose a subset of features:

1. **Subset selection:** Identify a subset of *k* features we believe are related to the response (e.g. stepwise selection).

2. **Shrinkage / regularization:** Fit a model with all *p* feature while shrinking the estimated coefficients towards 0 (e.g. Ridge Regression, Lasso).

3. **Dimensionality reduction:** Project *k* features into an M-dimensional subspace where $M < k$. We will cover this in-depth in a future lecture.

Subset Selection

Subset selection: Identify a subset of *k* features we believe are related to the response. This can be done several ways:

- **Exhaustive search:** search over every possible combination of features and check the performance of each model.
  - Can be very computationally expensive, we have to search through 2^m subsets.
- **Stepwise selection:**
  - **Forward stepwise selection:** start with the smallest possible model and add features one at a time.
  - **Backward stepwise selection:** start with the largest possible model (all features) and remove features one at a time.

One limitation is that this assumes that the features are independent.

Regularization

Unlike **subset selection, regularization** keeps all features but constrains the size of the coefficient estimates. By shrinking the coefficient estimates, their **variance can be reduced.**

3 popular methods for this:

1. **Ridge Regression (L_2 Regularization)**
2. **Lasso Regression (L_1 Regularization)**
3. **ElasticNet (Hybrid)**

Ridge Regression

Ridge Regression (L_2 Regularization): penalizes large coefficients (model parameters) using the L_2-norm. We add a penalty term:

$$\lambda \sum \beta_j^2$$

λ is the **regularization strength.** It controls how harshly you penalize big coefficients:

- If $\lambda = 0 \rightarrow$ no penalty, same as ordinary linear regression (as an example).
- If λ is large $\rightarrow$ stronger penalty, coefficients get pulled closer to zero.

Note that the L_2 penalty **shrinks coefficients towards zero** but never to absolute zero.

Lasso Regression

Lasso Regression (L_1 Regularization): similar to ridge regression, but using the L_1 norm. Our penalty term is now:

$$\lambda \sum |\beta_j|$$

As with Ridge, λ is the regularization strength. A smaller λ will impose a weaker penalty (closer to the linear regression coefficients), whereas a larger λ will shrink them closer to 0.

Note that since lasso is able to **shrink coefficients to 0**, it can **perform feature selection** and remove features completely.

ElasticNet

ElasticNet (Hybrid) combines the penalties of **ridge regression and lasso** to get the best of both worlds. The penalty term is:

$$\lambda \left(\alpha \sum |\beta_j| + (1 - \alpha) \sum \beta_j^2 \right)$$

λ = regularization strength

α = mixing parameter between ridge and lasso.

- α = 0 is equivalent to ridge regression
- α = 1 is equivalent to lasso

One downside is that we have to tune 2 parameters (λ and α) instead of one.

How do we Select λ?

Typically, **cross-validation** is used to select λ.

1. Choose a range of candidate λ values.
2. For each λ:
   a. Train the model on $k - 1$ folds of the data.
   b. Evaluate it on the remaining fold.
3. Compute the **average validation error** across folds for **each** λ.
4. Choose the λ that gives the **lowest validation error.**

How do we Select λ?

Note that in sklearn, the functions `RidgeCV`, `LassoCV`, and `ElasticNetCV` tune λ (and α in the case of ElasticNet) automatically.

Examples

```
>>> from sklearn.datasets import load_diabetes
>>> from sklearn.linear_model import RidgeCV
>>> X, y = load_diabetes(return_X_y=True)
>>> clf = RidgeCV(alphas=[1e-3, 1e-2, 1e-1, 1]).fit(X, y)
>>> clf.score(X, y)
0.5166...
```

See the sklearn docs for RidgeCV, LassoCV, and ElasticNetCV.

Scaling Before Regularization

Since regularization depends directly on the **magnitude of the coefficients**, it is very important to **standardize data before performing regularization!**

Example: Suppose you have 2 features: age (0-100) and income (30k-150k). To compensate for the difference in units, the coefficient for income will be much smaller than for age. If regularization is then applied, age get penalized much more, even though that difference is just due to unit scaling, not importance.

Standardization

We can standardize our data by subtracting the mean and dividing by the standard deviation:

$$x'_j = \frac{x_j - \bar{x}_j}{s_j}$$

This will ensure that all features have mean 0 and variance 1. This is equivalent to the `StandardScaler` function in sklearn.

Note that there are other methods to standardize data, but `StandardScaler` is one of the most commonly used ones.

Summary

- We can decompose generalization error into 3 parts: bias, variance, and irreducible error.
- Ideally, we want a model which **minimizes both bias and variance**, but these are conflicting goals. We aim to find the **best balance** between the 2.
- We can visually detect when a model has high bias or high variance by plotting a **learning curve**.
- To mitigate the effects of the bias-variance tradeoff, we can use techniques such as **feature selection** and **regularization**.
- It is very important to **standardize** data before applying regularization.

Further Resources

- Bias and Variance StatsQuest
- Cross-Validation StatsQuest
- Ridge (L2) Regularization
- Lasso (L1) Regularization
- Cornell - Bias Variance Tradeoff